Conclusion

Conclusion I

This study demonstrated a complete, reproducible exploratory-data-analysis pipeline driven from a computational notebook backed by a tested library. It validates a simple proposition: a notebook stays trustworthy when its cells call tested code instead of carrying logic.

Exemplar achievements

Exemplar achievements I

Operating as the EDA exemplar for the Research Project Template methodology, the project deployed the three foundational pillars:

Exemplar achievements I

1. `src/template_eda_notebook/eda/` **library**: pure pandas/numpy data transforms — loading, cleaning, summary statistics, correlation, and figure-data preparation — with no plotting, no file I/O, and no `infrastructure` imports.
2. `tests/` **integrity**: a zero-mock suite over the shipped dataset and hand-built frames, plus a structural notebook-binding check, all under a $\geq$90% project coverage gate.
3. `docs/` **knowledge operations**: architecture, testing philosophy, and operational rules that keep the notebook, script, and manuscript aligned.

Technical contributions

The notebook -> tested src extraction workflow I

The hallmark of this exemplar is the discipline it teaches: explore fast in a cell, and the moment a computation matters, move it into `src/template_eda_notebook/eda/` behind a failing test. The walkthrough notebook imports only the library's public surface and defines no functions of its own, a property the test suite enforces structurally so it cannot regress.

Honest handling of imperfect data I

The loader surfaces missing values as `NaN` instead of silently imputing them, and `clean_dataset()` removes incomplete rows with an explicit count. This makes data quality a visible, testable property of the first pass rather than a hidden assumption.

Key insights

Key insights I

1. **Reproducibility follows from testing fidelity**: every number in sec. 4 is produced by a tested function and regenerated on demand, so prose and artifacts cannot drift.
2. **Purity enables reuse**: because `src/template_eda_notebook/eda/` returns plot-ready data and never touches a display backend, the same functions serve the notebook, the headless script, and the manuscript.
3. **Missingness is information**: reporting dropped rows beats imputing them away in an exploratory pass.

Future extensions

Future extensions I

This foundation could be extended to:

Future extensions I

- **Richer cleaning**: typed imputation strategies behind tested functions, with the report recording which strategy ran.
- **More figure preparers**: box plots, pair plots, and per-group overlays — each a tested data preparer plus a thin plotting call.
- **Larger / external datasets**: swap the shipped CSV for a real dataset by pointing `load_dataset(path=...)` at it while keeping the schema contract.

Final assessment

Final assessment I

The `template_eda_notebook` tree is the canonical reference for how an exploratory notebook, a thin analysis script, a tested library, and a manuscript stay synchronized across rebuilds. The pipeline produced the figures referenced in sec. 4, wrote `output/data/summary_statistics.csv`, and rendered this markdown together with `config.yaml` into PDF.